マスタ管理画面UI設計書

メタデータ

title: マスタ管理画面UI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-01-23
author: Development Team
tags: [data-admin, master, list, main, creator, vendor, instrument]

関連ドキュメント

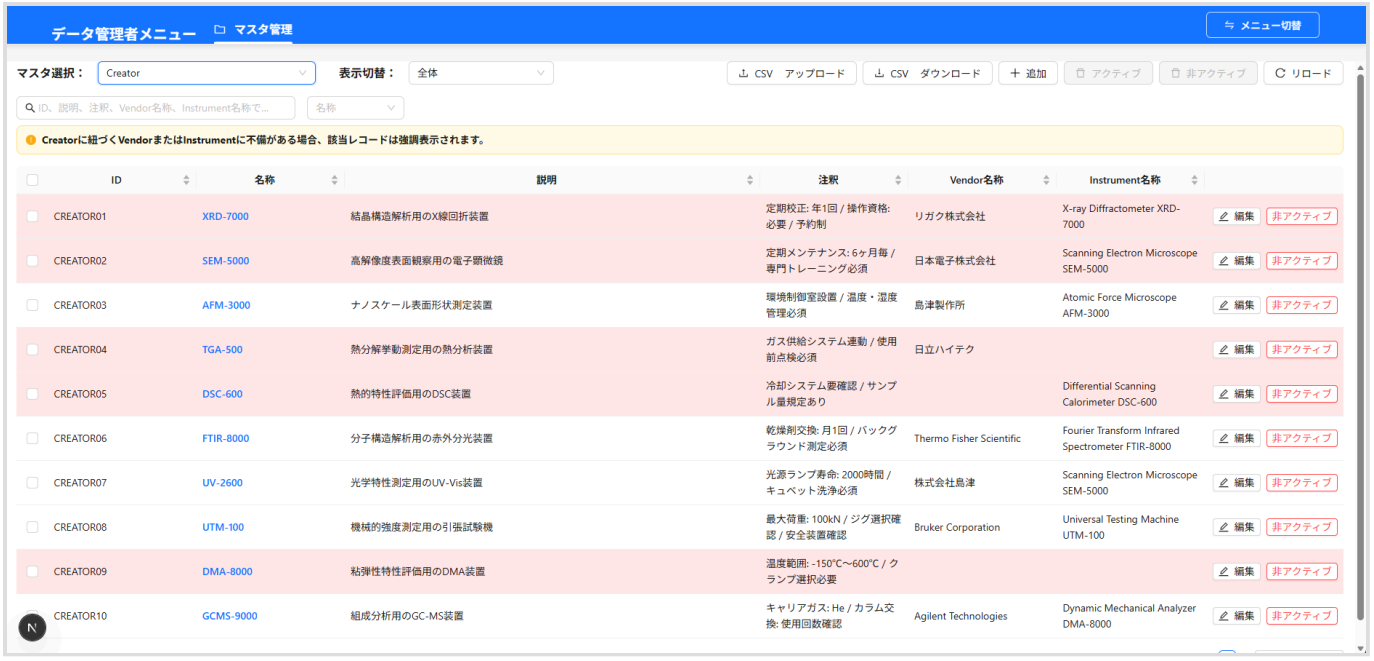
- マスタ追加ダイアログ
- マスタ編集ダイアログ
- マスタ管理API仕様
- CreatorテーブルDDL
- VendorテーブルDDL
- InstrumentテーブルDDL
- バックエンド実装計画
- UIポリシー

---

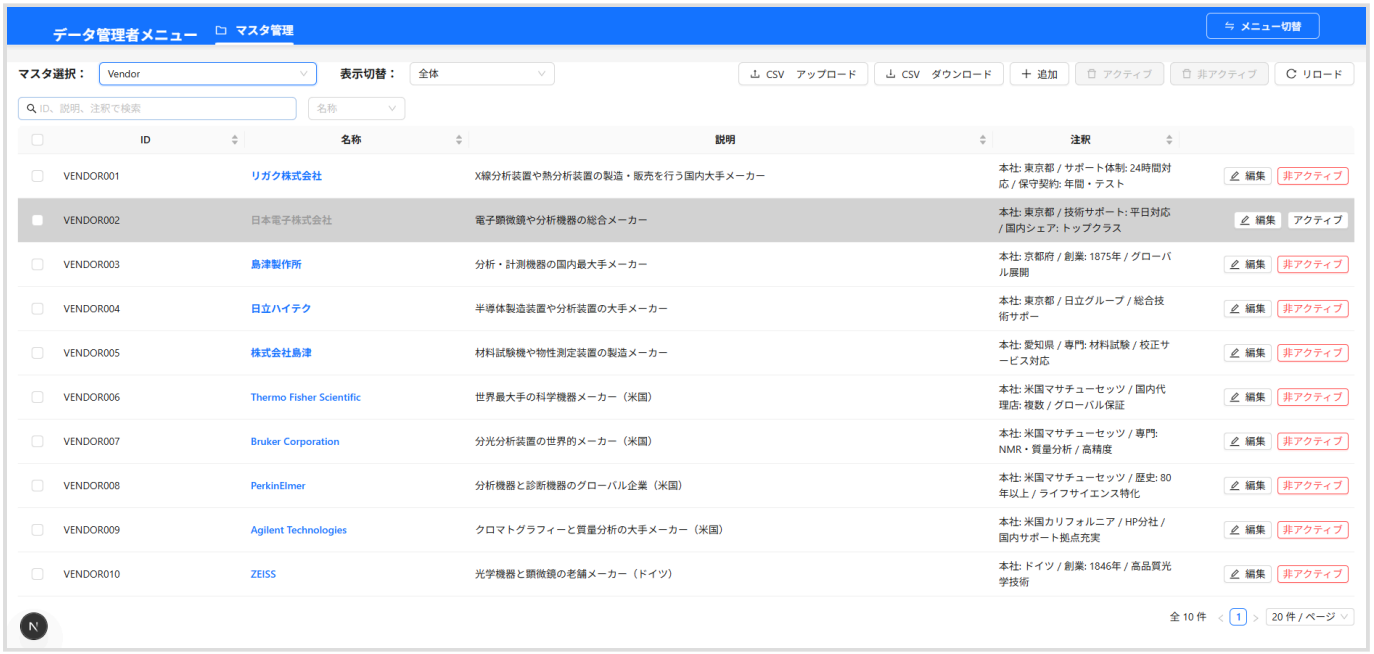
1. 画面レイアウト

1.1. 画面イメージ

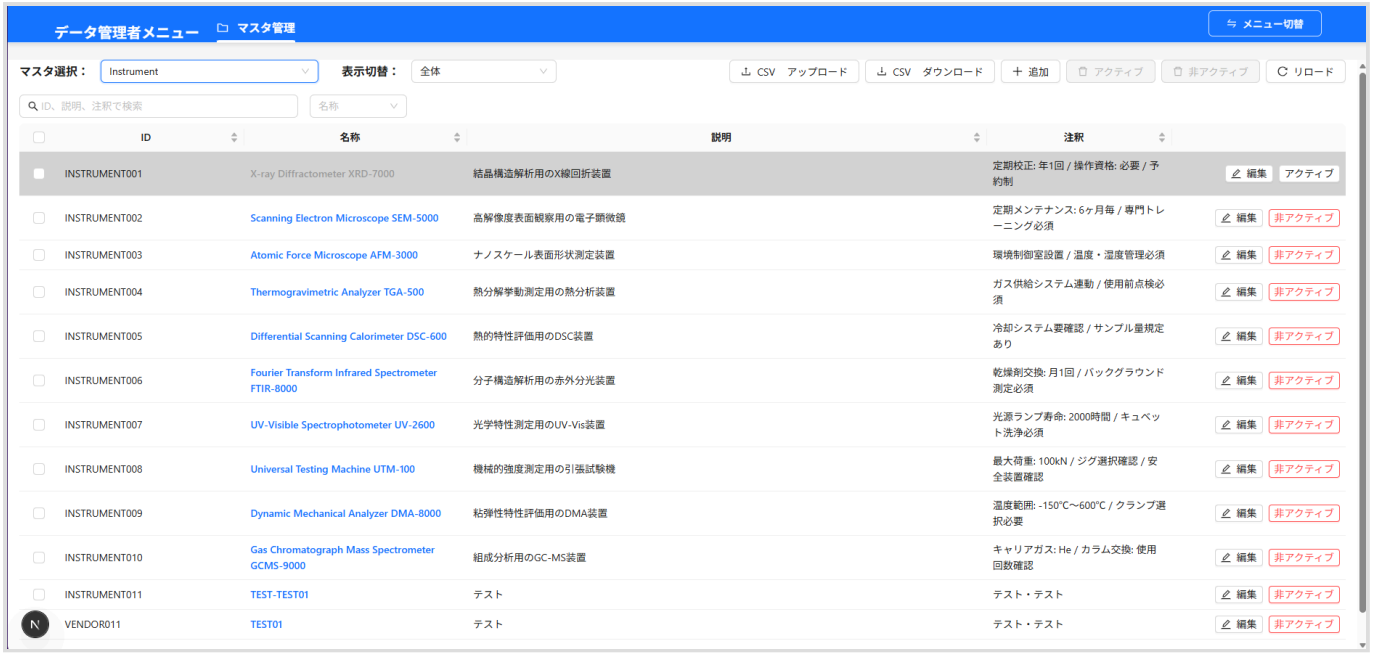
Creatorタブ:



Vendorタブ:



Instrumentタブ:



2. 概要

2.1. 目的

データ管理者が3種類のマスタデータ（Creator、Vendor、Instrument）を管理するためのメイン画面を定義します。タブ切り替えで各マスタの一覧表示、検索、フィルタリング、追加、編集、ステータス切替、CSV入出力機能を提供します。

2.2. 利用権限

- **Data Admin（データ管理者）**: 使用可能

2.3. URL

`/data-admin/master`

3. 画面構成

3.1. レイアウト

```
┌──────────────────────────────────────────────────────────┐
│ データ管理者 > マスタ管理                                │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  [Creator] [Vendor] [Instrument]  ← タブ                 │
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │ [🔍 検索] [▼ ステータス] [CSV⬇] [CSV⬆] [+ 追加]  │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │ ID ↕ │ 名前 ↕ │ 状態 ↕ │ 更新日時 ↕ │操作│      │    │
│  ├──────────────────────────────────────────────────┤    │
│  │ CRT001 │Creator A│ ●有効  │2026/01/23 │[編][切]│    │
│  │ CRT002 │Creator B│ ●有効  │2026/01/22 │[編][切]│    │
│  │ CRT003 │Creator C│ ○無効  │2026/01/21 │[編][切]│    │
│  │ ...                                              │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  全 25 件                              [< 1 2 3 4 >]     │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

---

4. タブ構成

4.1. タブ一覧

コンポーネント: `<Tabs>`

タブ項目:

タブキー	ラベル	マスタ種別
creator	Creator	作成者マスタ
vendor	Vendor	ベンダーマスタ
instrument	Instrument	実験装置マスタ

実装例:

```
const tabs = [
  {
```

```
    key: 'creator',
    label: 'Creator',
    children: <MasterTable masterType="creator" />
  },
  {
    key: 'vendor',
    label: 'Vendor',
    children: <MasterTable masterType="vendor" />
  },
  {
    key: 'instrument',
    label: 'Instrument',
    children: <MasterTable masterType="instrument" />
  }
];
```

---

5. ツールバー

5.1. 配置

左側: 検索・フィルタUI **右側**: CSV操作、追加ボタン（右から主要な順）

5.2. コンポーネント

5.2.1. 検索入力

コンポーネント: `<Input>`

仕様:

項目	値
プレースホルダー	「ID、名前で検索」
前置アイコン	`<SearchOutlined />`
幅	250px
クリアボタン	有効（allowClear）

検索対象（マスタ種別ごと）:

Creator:

- Creator ID（部分一致）
- Creator名（部分一致）

Vendor:

- Vendor ID（部分一致）
- Vendor名（部分一致）

Instrument:

- Instrument ID（部分一致）
- Instrument名（部分一致）

5.2.2. ステータスフィルター

コンポーネント: `<Select>`

仕様:

項目	値
プレースホルダー	「ステータス」
幅	150px
クリアボタン	有効（allowClear）

選択肢:

- すべて（フィルタなし）
- アクティブ
- 非アクティブ

5.2.3. CSVダウンロードボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「CSV」

タイプ	`default`
アイコン	`<DownloadOutlined />`
サイズ	省略（デフォルト）

動作:

1. 現在のフィルタ・検索条件を適用
2. API呼び出し: `GET /api/v1/master/{masterType}/download-csv`
3. CSVファイルをダウンロード
4. ファイル名: `{masterType}_YYYYMMDD_HHmmss.csv`

5.2.4. CSVアップロードボタン

コンポーネント: `<Upload>` + `<Button>`

仕様:

項目	値
ラベル	「CSV一括登録」
タイプ	`default`
アイコン	`<UploadOutlined />`
サイズ	省略（デフォルト）
許可形式	.csv

動作:

1. CSVファイル選択
2. アップロード前にプレビュー表示（オプション）
3. API呼び出し: `POST /api/v1/master/{masterType}/upload-csv`
4. 結果表示（成功件数、失敗件数、エラー詳細）

5.2.5. 追加ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「追加」
タイプ	`primary`
アイコン	`<PlusOutlined />`
サイズ	省略（デフォルト）

動作:

1. マスタ追加ダイアログを開く
2. 現在のマスタ種別を渡す
3. ダイアログでマスタ情報を入力
4. 保存後、一覧を再取得

---

6. データテーブル

6.1. テーブル仕様

コンポーネント: `<Table>`

主要設定:

項目	値
rowKey	マスタ種別に応じて変動（creatorId / vendorId / instrumentId）
pagination	有効
pageSize	20（デフォルト）
pageSizeOptions	[10, 20, 50, 100]
scroll	`{ x: 1200 }`

6.2. カラム定義（マスタ種別ごと）

6.2.1. Creator マスタ

Creator ID

プロパティ:

項目	値
title	「Creator ID」
dataIndex	`creatorId`
key	`creatorId`
width	150px
align	`left`
sorter	有効

Creator名

プロパティ:

項目	値
title	「Creator名」
dataIndex	`creatorName`
key	`creatorName`
width	250px
align	`left`
sorter	有効

状態

プロパティ:

項目	値
title	「状態」
dataIndex	`isActive`
key	`isActive`
width	120px
align	`center`
sorter	有効

レンダリング:

```
render: (isActive: boolean) => (
  <Badge
    status={isActive ? 'success' : 'default'}
    text={isActive ? 'アクティブ' : '非アクティブ'}
  />
)
```

更新日時

プロパティ:

項目	値
title	「更新日時」
dataIndex	`updateAt`
key	`updateAt`
width	180px
align	`right`
sorter	有効
defaultSortOrder	`descend`

レンダリング:

```
render: (date: string) => {
  return date ? new Date(date).toLocaleString('ja-JP') : '-';
}
```

操作

プロパティ:

項目	値
title	「操作」
key	`action`
width	200px
align	`center`

fixed	`right`

レンダリング:

```
render: (_, record) => (
  <Space size="small">
    <Button
      type="default"
      size="small"
      icon={<EditOutlined />}
      onClick={() => handleEdit(record.creatorId)}
    >
      編集
    </Button>
    <Button
      type="default"
      size="small"
      style={record.isActive ? {
        color: token.colorError,
        borderColor: token.colorError
      } : {
        color: token.colorSuccess,
        borderColor: token.colorSuccess
      }}
      onClick={() => handleStatusToggle(record.creatorId, !record.isActive)}
    >
      {record.isActive ? '非アクティブ化' : 'アクティブ化'}
    </Button>
  </Space>
)
```

6.2.2. Vendor マスタ

Vendor ID

プロパティ:

項目	値
title	「Vendor ID」
dataIndex	`vendorId`
key	`vendorId`
width	150px

align	`left`
sorter	有効

Vendor名

プロパティ:

項目	値
title	「Vendor名」
dataIndex	`vendorName`
key	`vendorName`
width	250px
align	`left`
sorter	有効

状態・更新日時・操作

Creatorと同様の構成

6.2.3. Instrument マスタ

Instrument ID

プロパティ:

項目	値
title	「Instrument ID」
dataIndex	`instrumentId`
key	`instrumentId`
width	150px
align	`left`
sorter	有効

Instrument名

プロパティ:

項目	値
title	「Instrument名」

dataIndex	`instrumentName`
key	`instrumentName`
width	250px
align	`left`
sorter	有効

Vendor ID

プロパティ:

項目	値
title	「Vendor ID」
dataIndex	`vendorId`
key	`vendorId`
width	150px
align	`left`
sorter	有効

状態・更新日時・操作

Creatorと同様の構成

---

7. ページネーション

7.1. 仕様

コンポーネント: `<Table>` の pagination プロパティ

設定:

```
pagination={{
  pageSize: 20,
  showSizeChanger: true,
  showTotal: (total) => `全 ${total} 件`,
```

```
  pageSizeOptions: ['10', '20', '50', '100']
}}
```

---

8. APIリクエスト仕様

8.1. 一覧取得

エンドポイント: `GET /api/v1/master/{masterType}`

マスタ種別パラメータ:

- `creator` - Creator マスタ
- `vendor` - Vendor マスタ
- `instrument` - Instrument マスタ

クエリパラメータ:

パラメータ	型	説明
search	string	検索テキスト（ID、名前）
is_active	boolean	アクティブフィルタ
sort_by	string	ソートフィールド
sort_order	string	ソート順（asc, desc）
page	number	ページ番号（デフォルト: 1）
limit	number	1ページあたりの件数（デフォルト: 20）

リクエスト例（Creator）:

```
GET /api/v1/master/creator?
search=Creator&is_active=true&sort_by=update_at&sort_order=desc&page=1&limit=20
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

レスポンス（Creator）:

```
{
  "success": true,
  "data": [
    {
      "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
      "creatorId": "CRT001",
      "creatorName": "Creator A",
      "isActive": true,
      "createAt": "2026-01-20T10:30:00.000Z",
      "updateAt": "2026-01-23T10:30:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalItems": 25,
    "totalPages": 2,
    "hasNext": true,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```


レスポンス（Vendor）：


```
{
  "success": true,
  "data": [
    {
      "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
      "vendorId": "VND001",
      "vendorName": "Vendor A",
      "isActive": true,
      "createAt": "2026-01-20T10:30:00.000Z",
      "updateAt": "2026-01-23T10:30:00.000Z"
    }
  ],
  "pagination": { "..." },
  "meta": { "..." }
}
```

レスポンス（Instrument）:

```
{
  "success": true,
  "data": [
    {
      "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
      "instrumentId": "INS001",
      "instrumentName": "Instrument A",
      "vendorId": "VND001",
      "isActive": true,
      "createAt": "2026-01-20T10:30:00.000Z",
      "updateAt": "2026-01-23T10:30:00.000Z"
    }
  ],
  "pagination": { "..." },
  "meta": { "..." }
}
```

8.2. ステータス更新

エンドポイント: `PATCH /api/v1/master/{masterType}/:id/status`

リクエストボディ:

```
{
  "isActive": false
}
```

レスポンス:

```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "creatorId": "CRT001",
    "creatorName": "Creator A",
    "isActive": false,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T11:00:00.000Z"
  },
  "meta": {
```

```
    "timestamp": "2026-01-23T11:00:00.000Z",
    "requestId": "req_def456"
  }
}
```

8.3. CSVダウンロード

エンドポイント: `GET /api/v1/master/{masterType}/download-csv`

クエリパラメータ: 一覧取得と同じ（search, is_active）

レスポンス: CSVファイル（UTF-8 BOM付き）

CSV形式（Creator）:

```
Creator ID,Creator名,状態
CRT001,Creator A,アクティブ
CRT002,Creator B,アクティブ
CRT003,Creator C,非アクティブ
```

CSV形式（Vendor）:

```
Vendor ID,Vendor名,状態
VND001,Vendor A,アクティブ
VND002,Vendor B,アクティブ
```

CSV形式（Instrument）:

```
Instrument ID,Instrument名,Vendor ID,状態
INS001,Instrument A,VND001,アクティブ
INS002,Instrument B,VND001,アクティブ
```

8.4. CSVアップロード

エンドポイント: `POST /api/v1/master/{masterType}/upload-csv`

リクエストヘッダー:

```
Content-Type: multipart/form-data
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

レスポンス:

```
{
  "success": true,
  "data": {
    "successCount": 18,
    "failureCount": 2,
    "errors": [
      {
        "row": 5,
        "id": "CRT999",
        "error": "IDが重複しています"
      },
      {
        "row": 12,
        "id": "",
        "error": "IDは必須です"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-23T11:00:00.000Z",
    "requestId": "req_ghi789"
  }
}
```

---

9. イベントハンドリング

9.1. タブ変更

トリガー: タブクリック

処理:

1. 選択されたマスタ種別を状態に保存

2. ページをリセット（page=1）
3. API呼び出し（新しいマスタ種別）

9.2. 検索

トリガー: 検索入力の変更（デバウンス500ms）

処理:

1. 検索テキストを状態に保存
2. 500msのデバウンス
3. API呼び出し（page=1にリセット）

9.3. ステータスフィルター

トリガー: ステータス選択の変更

処理:

1. 選択値を状態に保存
2. API呼び出し（page=1にリセット）

9.4. ソート

トリガー: カラムヘッダークリック

処理:

1. ソートフィールドとソート順を状態に保存
2. API呼び出し（page維持）

9.5. ページ変更

トリガー: ページネーションクリック

処理:

1. ページ番号を状態に保存
2. API呼び出し

9.6. 編集ボタンクリック

処理:

1. マスタIDとマスタ種別を保存
2. 編集ダイアログを開く
3. 保存後、一覧を再取得

9.7. ステータス切替ボタンクリック

処理:

1. 確認ダイアログ表示（非アクティブ化の場合）
2. API呼び出し: `PATCH /api/v1/master/{masterType}/:id/status`
3. 成功時、一覧を再取得
4. 失敗時、エラーメッセージ表示

---

10. 実装例（TypeScript + Next.js + Ant Design）

```
// frontend/src/app/data-admin/master/page.tsx
'use client';

import React, { useState, useEffect } from 'react';
import {
  Tabs,
  Table,
  Input,
  Select,
  Button,
  Space,
  Badge,
  message,
  Modal,
  Upload,
  theme
} from 'antd';
import {
  SearchOutlined,
  PlusOutlined,
```

```
  EditOutlined,
  DownloadOutlined,
  UploadOutlined
} from '@ant-design/icons';
import type { ColumnsType, TablePaginationConfig } from 'antd/es/table';
import apiClient from '@/utils/apiClient';
import MasterAddDialog from '@/components/DataAdmin/MasterAddDialog';
import MasterEditDialog from '@/components/DataAdmin/MasterEditDialog';

const { useToken } = theme;

type MasterType = 'creator' | 'vendor' | 'instrument';

interface MasterData {
  organizationId: string;
  creatorId?: string;
  creatorName?: string;
  vendorId?: string;
  vendorName?: string;
  instrumentId?: string;
  instrumentName?: string;
  isActive: boolean;
  createAt: string;
  updateAt: string;
}

export default function MasterManagementPage() {
  const { token } = useToken();
  const [activeTab, setActiveTab] = useState<MasterType>('creator');
  const [data, setData] = useState<MasterData[]>([]);
  const [loading, setLoading] = useState(false);
  const [searchText, setSearchText] = useState('');
  const [statusFilter, setStatusFilter] = useState<boolean | undefined>(undefined);
  const [sortField, setSortField] = useState('update_at');
  const [sortOrder, setSortOrder] = useState<'asc' | 'desc'>('desc');
  const [pagination, setPagination] = useState({
    current: 1,
    pageSize: 20,
    total: 0
  });

  const [isAddDialogOpen, setIsAddDialogOpen] = useState(false);
  const [isEditDialogOpen, setIsEditDialogOpen] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);

  const organizationId = '9eea5300-db8b-4b2d-8e9e-ec306c476b12'; // 実際は認証情報から取得

  // データ取得
  const fetchData = async () => {
    setLoading(true);
```

```
    try {
      const response = await apiClient.get(`/api/v1/master/${activeTab}`, {
        params: {
          search: searchText || undefined,
          is_active: statusFilter,
          sort_by: sortField,
          sort_order: sortOrder,
          page: pagination.current,
          limit: pagination.pageSize
        },
        headers: {
          'x-organization-id': organizationId
        }
      });

      if (response.data.success) {
        setData(response.data.data);
        setPagination({
          ...pagination,
          total: response.data.pagination.totalItems
        });
      }
    } catch (error: any) {
      message.error('データの取得に失敗しました');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchData();
  }, [activeTab, searchText, statusFilter, sortField, sortOrder, pagination.current,
pagination.pageSize]);

  // タブ変更
  const handleTabChange = (key: string) => {
    setActiveTab(key as MasterType);
    setPagination({ ...pagination, current: 1 });
    setSearchText('');
    setStatusFilter(undefined);
  };

  // 検索（デバウンス）
  useEffect(() => {
    const timer = setTimeout(() => {
      setPagination({ ...pagination, current: 1 });
    }, 500);
    return () => clearTimeout(timer);
  }, [searchText]);
```

```
// ステータス切替
const handleStatusToggle = (id: string, newStatus: boolean) => {
  const action = newStatus ? 'アクティブ化' : '非アクティブ化';
  Modal.confirm({
    title: `マスタを${action}しますか？`,
    content: `${id} を${action}します。`,
    okText: 'はい',
    cancelText: 'いいえ',
    onOk: async () => {
      try {
        await apiClient.patch(
          `/api/v1/master/${activeTab}/${id}/status`,
          { isActive: newStatus },
          {
            headers: {
              'x-organization-id': organizationId
            }
          }
        );
        message.success(`マスタを${action}しました`);
        fetchData();
      } catch (error: any) {
        message.error(`${action}に失敗しました`);
      }
    }
  });
};

// CSVダウンロード
const handleDownloadCsv = async () => {
  try {
    const response = await apiClient.get(
      `/api/v1/master/${activeTab}/download-csv`,
      {
        params: {
          search: searchText || undefined,
          is_active: statusFilter
        },
        headers: {
          'x-organization-id': organiz
```